Universidad Veracruzana

Facultad de Contaduría y Administración
Región Coatzacoalcos-Minatitlán

Programa educativo
Ingeniería de Software

Experiencia Educativa
Proyecto Guiado

Anteproyecto de Investigación

Ciclo de Vida, Marcos de Trabajo y Prácticas de la Ingeniería de Software en la era de la IA

Presentan:
Abraham Giovani Barrera Torres

Grupo:
LIS-701

Docente:
Dra. Ma. Teresa de la Luz Sainz Barajas

Coatzacoalcos, Ver., 9 de noviembre de 2026

"Lis de Veracruz: Arte, Ciencia, Luz"

Índice

Introducción 3
1. Fundamentos 4
    1.1. Planteamiento del problema 4
    1.2. Objetivo de la RSL 6
        1.2.1. **Objetivo General:** 6
        1.2.2. **Objetivos Específicos:** 6
    1.3. Pregunta de investigación 6
    1.4. Justificación 7
2. Método 7
    2.1. Criterios de inclusión y exclusión 7
        2.1.1. Criterios de inclusión 7
        2.1.2. Criterios de exclusión 7
    2.2. Proceso de búsqueda 8
    2.3. Evaluación de calidad 10
    2.4. Análisis de datos 16
3. Resultados 18
4. Conclusiones 21
5. Recomendaciones 22
Bibliografía 23

Introducción

El presente trabajo es un reporte de revisión sistemática de la literatura (RSL) desarrollado bajo las directrices metodológicas de Kitchenham y Charters. La investigación aborda la transformación de la ingeniería de software ante la integración de agentes de codificación y modelos de inteligencia artificial generativa en el ciclo de vida del desarrollo. Se examinarán las metodologías propuestas que integran IA en el desarrollo de proyecto de software y cómo cambian las dinámicas de trabajo y las competencias de trabajo..

1. Fundamentos

1.1. Planteamiento del problema

Desde sus inicios en 1968 en la OTAN, los modos y técnicas de hacer Software han evolucionado debido a la investigación de nuevos métodos y sobre todo a las tecnologías e infraestructura de cómputo. También es cierto que hay principios que permanecen vigentes y que fundamentan el enfoque sistemático del desarrollo. Para Chang (2026), la abstracción, el refinamiento sucesivo, divide y vencerás, y, a favor de hacer software para el ciudadano común, la integración de los resultados de investigación científica en software interdisciplinar (Internet de las Cosas, algoritmos genéticos, etc.). Es así como la Ingeniería de Software ha tenido cambios constantes que se han dirigido desde un enfoque en el hardware hacia una visión centrada en el usuario. El software ya no es solamente un sistema de uso científico o ingenieril, sino que se ha convertido en un producto de uso cotidiano accesible para el público general.

Por otro lado, la historia de la inteligencia artificial como disciplina también ha tenido avances cada vez más rápidos en los campos de deep learning y redes neuronales profundas. Pero fue en el año de 2024 cuando aumentaron las herramientas y tecnologías potenciadas por asistentes de codificación que el uso de inteligencia artificial en el desarrollo de software cobró relevancia. Los agentes de IA ya eran capaces de interactuar con herramientas de terceros, realizar búsquedas en la web, y actuar sobre el entorno digital, dejando de ser una herramienta secundaria a un componente activo en el desarrollo. Para 2025, el científico Andrej Karpathy hizo uso del concepto de “Vibe Coding” para referirse a la generación de código mediante lenguaje natural.

Como ingeniería, siempre ha sido de interés el aumentar la eficiencia en el desarrollo de proyectos. Primariamente, las dinámicas en los equipos de trabajo, los procesos y metodologías como Scrum han agilizado los tiempos de los proyectos, pero el uso de inteligencia artificial ha mejorado considerablemente el ritmo de trabajo. Kshetri & Voas (2026) señalan que se han observado tiempos de desarrollo que se reducen de meses a semanas o días al examinar los resultados del vibe coding en proyectos reales. Los equipos de trabajo pueden dedicar más tiempo a la innovación, prototipos y lógica de negocio debido a la automatización de generación de código.

Frente a esta situación, quienes ignoran el uso de la inteligencia artificial se encuentran en posición de desventaja para hacer frente a las necesidades del desarrollo moderno, que exige tiempos y dinámicas más rápidas. Si el vibe coding resulta ahora indispensable para el desarrollo, entonces el problema es reconocer las nuevas prácticas, habilidades, procesos y metodologías que respondan eficientemente a las tendencias actuales.

A partir de esta transición del desarrollo tradicional hacia un modelo fundamentado en la orquestación mediante agentes autónomos de inteligencia artificial, surge la necesidad de replantear los principios actuales de la disciplina. Este trabajo pretende determinar cuáles son las

metodologías, competencias críticas y criterios educativos que deben estructurar la formación y el desempeño de los ingenieros de software, que garanticen la efectividad operativa del desarrollo.

1.2. Objetivo de la RSL

1.2.1. Objetivo General:

Analizar los conocimientos, tecnologías, competencias y criterios de formación académica que requieren los ingenieros de software ante la tendencia de integrar inteligencia artificial en el desarrollo de proyectos software.

1.2.2. Objetivos Específicos:

1. Identificar las prácticas actuales de diseño de arquitectura y desarrollo de sistemas de los ingenieros de software potenciadas con el uso de IA.
2. Examinar la comunicación, pensamiento crítico y trabajo colaborativo de los ingenieros de software en un contexto de trabajo asistido por IA.
3. Determinar las metodologías de trabajo y procesos que optimizan los recursos de un proyecto de desarrollo de software mediante la orquestación de agentes de IA.
4. Analizar cómo se estructuran las etapas del ciclo de vida del software en los modelos que integran IA.
5. Identificar las prácticas de validación y aseguramiento de la calidad que emplean los ingenieros de software para mitigar los riesgos del código generado por agentes de IA.

1.3. Pregunta de investigación

1. ¿Cuáles son las prácticas de los ingenieros de software en el diseño arquitectónico de sistemas cuando el proceso de desarrollo es dirigido por IA?
2. ¿De qué manera la integración de la IA generativa redefine la comunicación técnica, el pensamiento crítico y la colaboración en los equipos de desarrollo de software?
3. Si el desarrollo se lleva a cabo con el uso de agentes de IA, ¿cuáles son las metodologías de trabajo y procesos que resultan más efectivos para optimizar el desarrollo?
4. ¿Cómo se estructuran las etapas del ciclo de vida de software con la integración de IA?
5. ¿Qué prácticas de validación y aseguramiento de la calidad emplean los ingenieros de software para mitigar los riesgos del código generado por agentes de IA?

1.4. Justificación

La industria de desarrollo de software actual demanda tiempos de entrega cada vez más rápidos y cortos plazos de tiempo, lo cual implica presión que generan ciertos conflictos que afectan el proyecto. Kuutila et al. (2017) indican que los problemas asociados con la presión en tiempos de entrega incluyen una mayor probabilidad de cometer errores, mayor número de defectos, trabajo posterior al despliegue, falta de coordinación en los equipos de trabajo, etc.

En el panorama actual, la construcción de software utilizando IA generativa ha significado una contribución importante para acelerar los proyectos y mejorar los tiempos de entrega. Evaluaciones realizadas por Srinivas et al. (2026), tras analizar 50 proyectos de software, se encontró que un framework impulsado por IA generativa logró la reducción de 50.6% en el tiempo de desarrollo en todo el ciclo de vida del software.

La investigación actual tiene como propósito determinar las tendencias actuales de construcción de software y se justifica debido a implicaciones relevantes para con las exigencias de la industria del desarrollo. Debido al cambio de paradigma de la forma de hacer ingeniería de software las funciones de los ingenieros se han replanteado y es pertinente identificar formalmente cuáles son estas nuevas prácticas, metodologías, y enfoques actuales.

Esta investigación se considera viable ya que se fundamenta en la revisión documental de literatura científica publicada entre 2020 y 2026 procedente de las bases de datos IEEE, ACM Digital Library, Science Direct, Scopus y Arxiv.

2. Método

2.1. Criterios de inclusión y exclusión

2.1.1. Criterios de inclusión

1. Papers, capítulos de libros, y artículos de revistas publicados entre los años 2020 y 2026. (CI1)
2. Papers escritos en inglés. (CI2)
3. Papers con acceso completo. (CI3)
4. Título y el abstract puedan dar respuesta al menos una pregunta de investigación. (CI4)
5. Leyendo el contenido responde al menos una pregunta de investigación. (CI5)

2.1.2. Criterios de exclusión

1. En fuentes multidisciplinarias, el estudio no está en el área de Ingeniería o Computer Science. (CE1)

2. La publicación es una presentación o reporte técnico. (CE2)

3. Papers duplicados. (CE3)

4. Después de leer los paper, no aborda ninguna pregunta de investigación. (CE4)

2.2. Proceso de búsqueda

	Search	Stage 1	Stage 2	Stage 3
IEEE	2118	169	12	2
ACM Digital Library	231	220	12	1
Scopus	3829	531	9	2
ArXiv	1607	1559	22	2
Science Direct	633	234	9	0
Total	8418	2713	64	**7**

Tabla 1: Resultados de la cadena de búsqueda 1

	Search	Stage 1	Stage 2	Stage 3
IEEE	2656	261	24	1
ACM Digital Library	226	216	15	3
Scopus	1514	256	37	3
ArXiv	580	580	23	2
Science Direct	2826	1098	23	0
Total	7802	2411	122	**9**

Tabla 2: Resultados de la cadena de búsqueda 2

	Search	Stage 1	Stage 2	Stage 3
IEEE	825	28	9	2
ACM Digital Library	70	38	8	1
Scopus	406	63	12	1
ArXiv	8893	1071	26	2
Science Direct	15835	2486	14	0
Total	26029	3686	69	**6**

Tabla 3: Resultado de la cadena de búsqueda 3

	Search	Stage 1	Stage 2	Stage 3
IEEE	1995	206	48	1
ACM Digital Library	150	118	26	2
Scopus	1929	326	9	1
ArXiv	545	472	14	2
Science Direct	911	357	57	1
Total	5530	1479	154	**7**

Tabla 4: Resultado de la cadena de búsqueda 4

	Search	Stage 1	Stage 2	Stage 3
IEEE	1242	83	8	0
ACM Digital Library	300	275	44	0
Scopus	1264	238	18	0
ArXiv	739	432	17	1
Science Direct	6731	1780	5	0
Total	10276	2808	92	**1**

Tabla 5: Resultado de la cadena de búsqueda 5

2.3. Evaluación de calidad

Pregunta	Respuesta
¿Están claramente especificados los objetivos de la investigación?	Y/N/P
¿El estudio fue diseñado para lograr objetivos?	Y/N/P
¿Se describen claramente las técnicas utilizadas y se justifica su selección?	Y/N/P
¿Se miden adecuadamente las variables consideradas por el estudio?	Y/N/P
¿Se describen adecuadamente los métodos de recopilación de datos?	Y/N/P
¿Se describen adecuadamente los datos recopilados?	Y/N/P
¿Es claro el propósito del análisis de datos?	Y/N/P
¿Se utilizan técnicas estadísticas para analizar los datos adecuadamente descritos y su uso está justificado?	Y/N/P
¿Se presentan resultados negativos (si los hay)?	Y/N/P
¿Los investigadores discuten algún problema con la validez/confiabilidad de los resultados?	Y/N/P
¿Se responden adecuadamente todas las preguntas de investigación?	Y/N/P
¿Qué tan claros son los vínculos entre los datos, la interpretación y las conclusiones?	Y/N/P

Tabla 6: Preguntas para la evaluación de calidad

Para la evaluación de la calidad de las fuentes se tomaron en cuenta los criterios de Kitchenham. Se hizo especial énfasis en las preguntas de calidad 3,4 y 5 sobre la recopilación de datos. Aquellas fuentes que tenían una puntuación menor a 2 eran fuentes de índole secundaria o propuestas teóricas sin datos empíricos.

ID	Título del Artículo	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	P
1	A-SAW: The Agentic Software Architecture Workflow	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
2	A Comprehensive Framework for Intelligent, Scalable, and Performance-Optimized Software Development	Y	Y	P	P	N	N	P	P	Y	N	N	N	4.5

ID	Título del Artículo	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	P
3	A Framework for Evaluating GenAI Adoption and Use in Software Engineering	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
4	Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence, and the Reshaping of Software Engineering	Y	Y	N	N	N	P	Y	N	Y	N	P	Y	6.0
5	Are We All Using Agents the Same Way An Empirical Study of Core and Peripheral Developers Use of Coding Agents													
6	Better Together: An LLM-Assisted Approach to Designing Software Architectures Using Attribute-Driven Design	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
7	CICAP: A Human-AI Co-creation Paradigm for Ethical and	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	N	Y	11.0

ID	Título del Artículo	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	P
	Efficient Plugin Development in Moodle													
8	Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows													
9	Command Coding: A Methodological Framework for AI-Accelerated Software Development	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
10	Copiloting the future: How generative AI transforms Software Engineering	Y	Y	Y	P	Y	Y	Y	Y	Y	P	Y	Y	11.0
11	Empowering Agile-Based Generative Software Development through Human-AI Teamwork	Y	Y	Y	Y	Y	Y	Y	N	Y	N	Y	Y	10.0
12	From Human--to-Human to Human-to-Bot Conversations in Software Engineering	Y	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y	10.0
13	From Prompting to Verification:	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0

ID	Título del Artículo	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	P
	How Experience Shapes Vibe Coding Practices													
14	From Disruptions to Discussions: How GenAI Impacts Human Interactions in Software Development	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
15	Generative AI for software architecture. Applications, challenges, and future directions	Y	Y	P	P	P	Y	Y	Y	Y	Y	Y	Y	10.5
16	Generative AI in Software Development: Challenges, Opportunities, and New Paradigms for Quality Assurance	Y	P	P	Y	P	Y	Y	N	Y	N	P	Y	8.0
17	How Do Developers Interact with AI? An Exploratory Study on Modeling Developer Programming Behavior	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
18	LLMs' reshaping of people, processes, products, and society in	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0

ID	Título del Artículo	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	P
	software development: a qualitative exploration with early adopters													
19	One Developer Is All You Need: A Case Study of an AI-Augmented One-Person Squad in a Brownfield Enterprise	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
20	Reframing Software Architecture and Design Skills in the Era of Generative Artificial Intelligence	Y	N	N	N	N	N	N	N	N	N	N	N	1.0
21	Rethinking Software Engineering for Agentic AI Systems	Y	Y	P	N	P	Y	Y	P	Y	Y	Y	Y	9.5
22	SDAD: Spec-Driven Agentic Development for the AI-Native SDLC	Y	Y	P	P	N	N	Y	P	Y	Y	Y	Y	8.5
23	Software Engineering by and for Humans in an AI Era	Y	Y	Y	N	N	N	Y	N	Y	N	Y	Y	7.0
24	The Impact of Modern AI on	Y	Y	P	P	P	Y	Y	P	Y	N	Y	Y	9.0

ID	Título del Artículo	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	P
	Software Development: A Systematic Literature Review													
25	The role of LLMs in Collaborative Software Design													
26	The Role of Software Engineering Knowledge in Shaping Effective AI-Driven Development													
27	Using LLMs in Software Design: An Empirical Study of GitHub and A Practitioner Survey	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	12.0
28	Vibe Coding as a Reconfiguration of Intent Mediation in Software Development: Definition, Implications, and Research Agenda	Y	Y	P	N	N	P	P	N	Y	P	Y	Y	6.0
29	What Will Remain Human in Software Archi-	Y	Y	Y	N	Y	Y	Y	N	Y	Y	Y	Y	10.0

ID	Título del Artículo	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	P
	tecture? A Focus Group Report													
30	When Clean Code Becomes Costly: Balancing Human Readability and AI Digestibility with the NUDEE Framework	Y	Y	N	N	N	Y	Y	P	Y	Y	Y	Y	8.5

Tabla 7: Evaluación de la calidad QA

2.4. Análisis de datos

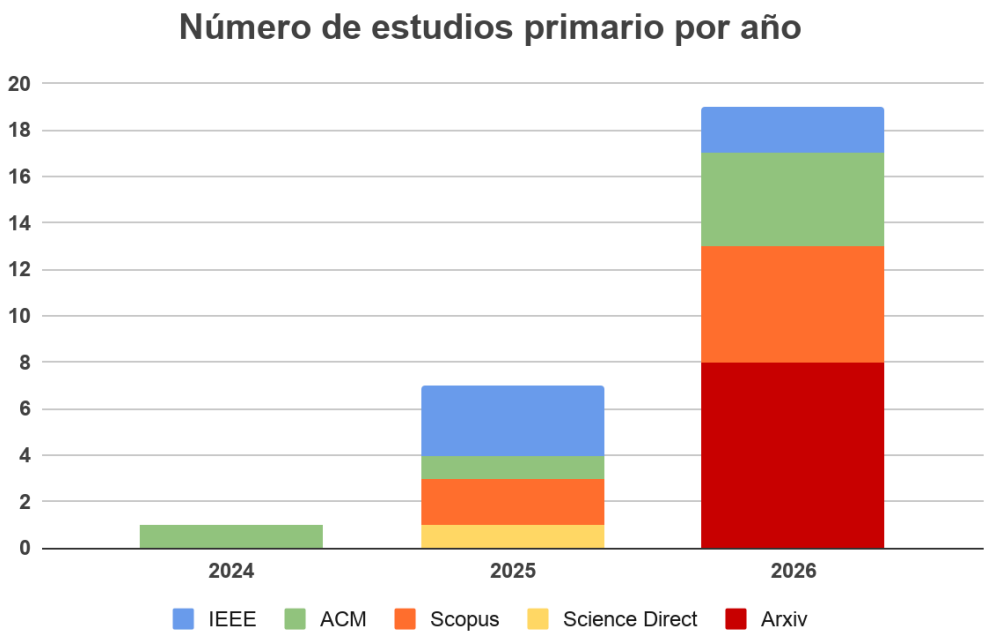


Figura 1: Número de artículos por año

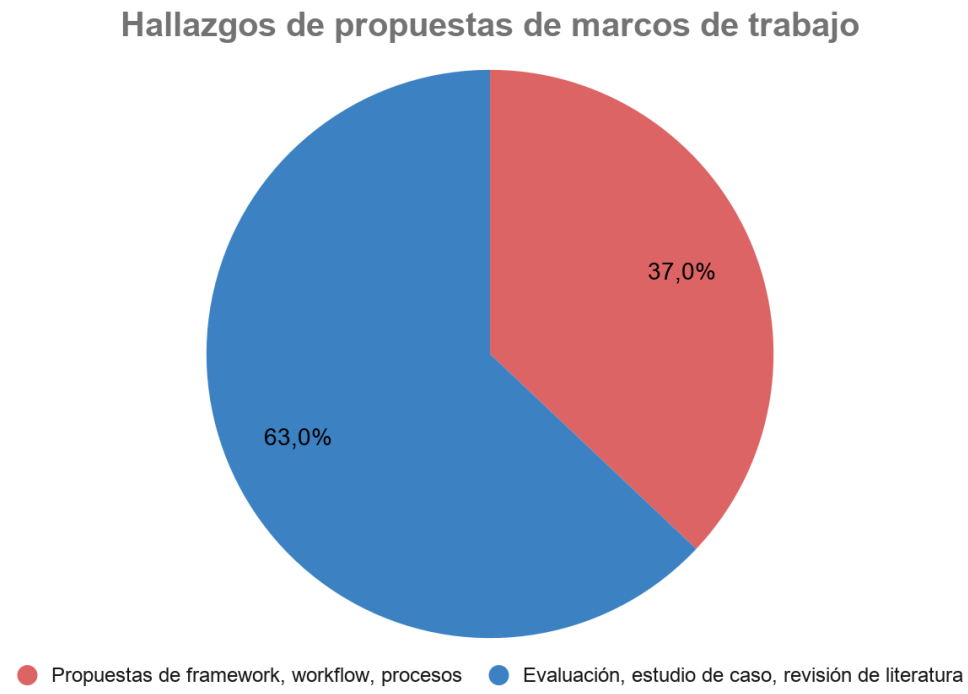


Figura 2: Tipos de investigación en fuentes primarias

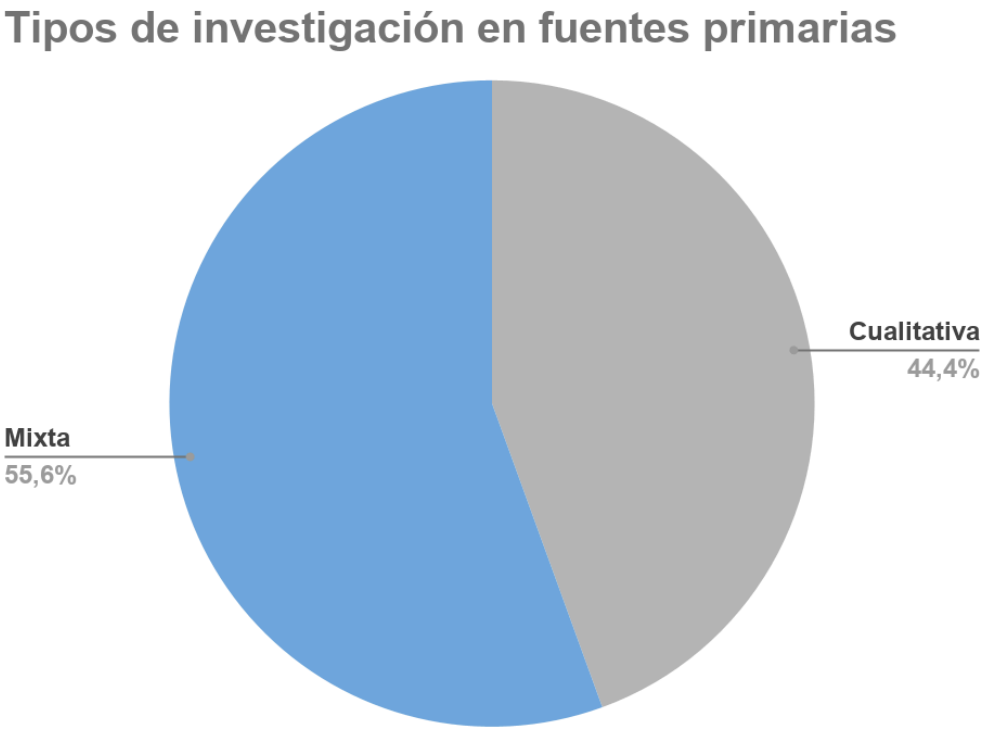


Figura 3: Proporción de estudios que proponen marcos de trabajo

3. Resultados

Si el desarrollo se lleva a cabo con el uso de agentes de IA, ¿cuáles son las metodologías de trabajo y procesos que resultan más efectivos para optimizar el desarrollo?

Villani & Kellogg (2026) evalúan una metodología propuesta denominada Command Coding que requiere habilidades nivel senior en arquitectura. La metodología fue evaluada en el desarrollo de la librería all2md; el proyecto fue completado en 41 días de trabajo y se estima que acelerar el trabajo entre 25 y 40 veces el ritmo de trabajo en comparación con el desarrollo tradicional sin pérdida de la calidad. Esto se logra mediante sprints pequeños diarios que desarrollan de 3 a 5 elementos de trabajo.

Por otra parte, Karroumi et al. (2026) evaluaron una metodología enfocada en la interacción humano-IA denominada Collaborative Iterative Co-Authoring Process (CICAP). Se evaluó este marco de trabajo en el desarrollo de un plugin para la plataforma Moodle y se observó una reducción del 60% del tiempo total estimado del proyecto. El flujo de trabajo consta de 6 etapas que incluye la documentación final de los prompts y su respuesta para la reproducibilidad y aprendizajes en proyectos futuros.

También la metodología Specification-Driven Development (SDD) es señalada por Boas et al. (2026) como un paradigma emergente en el desarrollo asistido por IA. Esta metodología se fundamenta en el desarrollo dirigido por pruebas y el objetivo principal es generar especificaciones que un agente de IA pueda implementar sin ambigüedad. En este caso, los autores evalúan el desarrollo de una plataforma de firma digital para un banco en el que un solo desarrollador configura los roles de varios agentes (Product Manager, Specification y Developers).

¿Cuáles son las prácticas de los ingenieros de software en el diseño arquitectónico de sistemas cuando el proceso de desarrollo es dirigido por IA?

En la revisión realizada por Wang et al. (2026), con base en una encuesta, se ha observado que los LLMs se han utilizado para actividades de diseño de alto nivel, descomposición modular, estructura organizacional, pros y contras, así como patrones de interacción y coordinación. Kang et al. (2026) indica que los ingenieros deben guiar a la IA para definir esquemas de datos y estructurar arquitecturas multicapa, de otra manera la IA genera componentes aislados que difícilmente pueden integrarse, esquemas con falta de coherencia y mecanismo de seguridad deficientes.

Heesch et al. (2026) resalta la importancia que tiene la especificación previo a la implementación por parte de agentes de IA, puesto que la calidad de la salida es directamente proporcional a la precisión de la entrada especificada. Así lo han reportado participantes en encuestas. También señala que algunos usuarios prefieren implementar algunas características de manera independiente para que el agente aprenda de manera más rápida el estilo del código.

Por su parte, Cervantes et al. (2026) describe las siguientes prácticas:

- **Método de Attribute Driven Design:** utiliza el método de arquitectura Architecture-Driven Design (ADD), esto es descomponer el diseño arquitectónico en pasos secuenciales para generar artefactos intermedios en el proceso.
- **Definición de rol:** configurar el perfil del LLM como arquitecto sénior que justifica cada propuesta por atributo de calidad, así como la evaluación de ventajas y desventajas.
- **Architectural drivers:** proporcionar los drivers arquitectónicos como entradas al LLM que incluyen requisitos funcionales primarios, escenarios de atributos de calidad, restricciones y decisiones arquitectónicas. Estos artefactos deben ser prioritarios.

Klatt et al. (2026) propone Agentic-Software Architecture Workflow (A-SAW) como un flujo de trabajo de tres capas (Artifact, Agent Orchestration, Human Review) para crear artefactos de arquitectura y reducir el esfuerzo de mantenerlos actualizados en las iteraciones del ciclo de vida del software.

¿Cómo se estructuran las etapas del ciclo de vida de software con la integración de IA?

Tabarsi et al. (2026) declara que los LLMs influyen en 5 fases del ciclo de vida del software. Se percibe que:

- **Requisitos:** menor adopción de IA.
- **Diseño:** uso de LLMs para la descomposición del problema.
- **Implementación:** uso de estrategias de prompts sofisticadas.
- **Pruebas y revisión de código:** pruebas unitarias con limitaciones.
- **Debugging, refactorización y documentación:** el uso de LLMs reduce el tiempo de debugging.

Para Muratovic et al. (2025) el ciclo de vida se puede estructurar en 4 fases integradas: la planificación y diseño que incluye la recolección de requisitos, arquitectura y prototipado; la implementación que abarca autocodificación, depuración y control de precisión; las pruebas generadas con IA y el despliegue y optimización que incluye la configuración de pipelines CI/CD, monitoreo e ingeniería financiar para gestionar costos computacionales.

Para Yu et al. (2026), el proceso de software se extiende previo a los requisitos de un proyecto. En este caso, la Ideación comienza desde la adopción de una IA generativa: identificar las ideas de negocio, inquirir en modelos de IA, probar prompts de prueba y la selección de modelos. Las siguientes etapas del ciclo de vida corresponden al desarrollo, y son bastante apegadas al modelo tradicional: requisitos, desarrollo, pruebas y validación. El último gran apartado es la Operación: despliegue, monitoreo, evaluar tiempo de ejecución y mejora de la aplicación. Siempre es posible volver entre cada gran apartado, por ejemplo para cambiar de modelo de IA o para ampliar el desarrollo.

¿De qué manera la integración de la IA generativa redefine la comunicación técnica, el pensamiento crítico y las dinámicas de colaboración en los equipos de desarrollo de software?

Boas et al. (2026) sugiere que la inteligencia artificial maximiza la tendencia de reducción de equipos de trabajo que requieren mayor coordinación, pero que disminuye la especialización que ocupa cada miembro. Mediante el uso de agentes de IA, un ingeniero puede trabajar sin sufrir la carga cognitiva de un equipo de trabajo y con los beneficios de las capacidades de un equipo de trabajo, asignando un rol a múltiples agentes de IA.

En este caso, se estructuran tres roles durante el ciclo de vida:

- **Product manager:** agente configurado con conocimiento del dominio del problema.
- **Agente de especificaciones:** con acceso a repositorios, microservicios, componentes, etc.
- **Desarrolladores:** divididos entre módulos críticos, como reglas de negocio y casos de uso, y módulos no críticos como integración de APIs, configuración de cola de mensajes y código boilerplate.

Según Wu et al. (2026), algunos desarrolladores aumentan su escepticismo con el uso de código generado por IA. Prefieren utilizar sólo porciones de código y escribir el resto manualmente porque es más fácil la verificación posterior.

Cuando se trata de la revisión de integraciones de código por medio de pull requests, Cynthia et al. (2026) ha encontrado que los desarrolladores principales tienden a priorizar soluciones alternativas frente a los problemas relacionados con código generado mientras que los desarrolladores secundarios se enfocan en la organización del código.

En el estudio realizado por Salomon et al. (2026) los desarrolladores Sénior prefieren que los desarrolladores novatos utilicen IA para responder preguntas triviales o rutinarias, y reservar las preguntas que requieren discusión y múltiples puntos de vista bajo la guía humana. Ante esto, los novatos declaran que esta dinámica resulta cómoda cuando se trata de preguntas que les apena consultar con los superiores. En este sentido, es importante la habilidad de delegar preguntas a la IA generativa para mantener la concentración en las tareas inmediatas y el ritmo de trabajo. Como resultado, se reportó un 59% de mejoría en los tiempos de las actividades y además algunos desarrolladores dicen haber incrementado la confianza en las soluciones. Khojah et al. (2024) declara que la comunicación humano-humano asume un contexto explícito e implícito compartido por el equipo, mientras que las comunicación con LLMs requiere personalización para la comprensión del problema.

Al trabajar en parejas (pair-programming), Jackson et al. (2026) encuentra tres patrones diferentes del uso de LLMs en las actividades. Los pares pueden elegir trabajar con un LLM compartido donde un desarrollador dirige y escribe los prompts y el segundo contribuye a construir y revisar la respuesta. La segunda forma de trabajo es utilizar dos instancias de LLM para una misma actividad; este enfoque permite explorar las capacidades no determinísticas de la IA pero puede causar problemas de desviación de contexto. Adicionalmente, los desarrolladores pueden

separar las actividades en subactividades para cada quien trabajar en su propia instancia; este modo de trabajo requiere mayor coordinación para asegurar la coherencia.

¿Qué prácticas de validación y aseguramiento de la calidad emplean los ingenieros de software para mitigar los riesgos del código generado por agentes de IA?

En la metodología de Command Coding Villani & Kellogg (2026) la revisión es continua y tiene tres escenarios:

- **Filtros automáticos de seguridad:** se ejecuta en cada commit el código, tipado, ejecución pruebas y escaneo de calidad.
- **Revisión entre modelos:** cada 3-5 elementos de trabajo un modelo diferente del que genera el código revisa el código, vulnerabilidades, consistencia de patrones y que sea coherente con la arquitectura.
- **Revisión humana:** la revisión de humanos se enfoca en elementos críticos de seguridad, decisiones en el diseño de APIs, alineación con la arquitectura y adecuación de pruebas.

El uso de IA generativa también puede utilizarse para la escritura de pruebas unitarias específicas pero también para la verificación de código recién escrito para encontrar errores lógicos, vulnerabilidades o casos de refactorización, así lo menciona Banh et al. (2025). Zhang et al. (2025) menciona además que los desarrolladores suelen solicitar explicaciones de las modificaciones de múltiples archivos, examinar diferencias de código y solicitar revisiones ante la sobre-generación de archivos o fallos.

Fawzy et al. (2026) agrega que los desarrolladores experimentados suelen optar por la corrección manual antes que pedirle a la IA que corrija su propio código. Para las evaluaciones, Zhang et al. (2025) propone casos de prueba técnicos basados en criterios de aceptación que utilizan lenguaje Gherkin.

4. Conclusiones

Finalmente, tras completar el proceso de búsqueda y recuperar información de los artículos seleccionados se pueden obtener las siguientes conclusiones respecto al ciclo de vida, marcos de trabajo y prácticas de los ingenieros de software al utilizar IA en los proyectos de desarrollo de software. En el diseño arquitectónico, los hallazgos muestran que la IA actúa como un facilitador para actividades de alto nivel y estructuración modular, aunque exige que los ingenieros asuman un rol riguroso de guía para evitar inconsistencias y componentes aislados; la aplicación de métodos estructurados como Attribute-Driven Design (ADD) y flujos como A-SAW resulta clave para priorizar atributos de calidad y mantener artefactos actualizados.

Respecto a las metodologías de trabajo y procesos más efectivos, marcos emergentes como Command Coding, CICAP y Specification-Driven Development (SDD) han demostrado optimizar drásticamente el ritmo de desarrollo y la aceleración de entrega, reduciendo tiempos hasta en un

60%. En las etapas del ciclo de vida del software, la adopción de IA engloba la ideación temprana, formulación de requisitos, despliegue, depuración y monitoreo continuo.

Por su parte, las dinámicas de colaboración y comunicación técnica han cambiado hacia modelos de interacción humano-bot, permitiendo la reducción en el tamaño de los equipos sin perder alcance de especialización entre miembros de un equipo, además se modifican las prácticas de pair-programming y la delegación de tareas rutinarias a la IA para priorizar el pensamiento crítico humano.

Finalmente, el aseguramiento de la calidad y la mitigación de riesgos asociados al código generado demandan un enfoque de validación en múltiples capas: filtros automáticos de seguridad en CI/CD, revisiones cruzadas entre distintos modelos de lenguaje y la supervisión humana experta enfocada en componentes críticos de arquitectura y seguridad. De esta manera, el nuevo paradigma promueve una ingeniería de software donde el valor principal del profesional radica en la especificación, verificación y dirección estratégica del sistema.

5. Recomendaciones

Para el desarrollo de esta Revisión Sistemática de Literatura se tomaron en cuenta las bases de datos IEEE, ACM, Scopus, Science Direct y Arxiv. Para el tema presentado en esta investigación la literatura gris no queda fuera del alcance, puesto que de acuerdo a las tendencias observadas en el análisis de datos la información respecto a metodologías es reciente, y Arxiv es una excelente herramienta para buscar información actualizada.

Bibliografía

Banh, L., Holldack, F., & Strobel, G. (2025). Copiloting the future: How generative AI transforms Software Engineering. *Inf. Softw. Technol.*, *183*(C). https://doi.org/10.1016/j.infsof.2025.107751

Boas, M. V., Pinto, G., Monteiro, E. R., Carida, V. F., & Ribeiro, D. (2026,). *One Developer Is All You Need: A Case Study of an AI-Augmented One-Person Squad in a Brownfield Enterprise*. https://arxiv.org/abs/2605.18461

Cervantes, H., Kazman, R., & Cai, Y. (2026). Better Together: An LLM-Assisted Approach to Designing Software Architectures Using Attribute-Driven Design. *IEEE Transactions on Software Engineering*, *52*(10), 2970-2985. https://doi.org/10.1109/TSE.2026.3706446

Chang, C. K. (2026). History and Future Trends of Modern Computing and Software Engineering – Personal Recounting of 80 Years of the IEEE Computer Society and 50 Years of IEEE COMPSAC. *2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC)*, 898-905. https://doi.org/10.1109/COMPSAC69091.2026.00123

Cynthia, S. T., Das, J. K., & Roy, B. (2026). Are We All Using Agents the Same Way? An Empirical Study of Core and Peripheral Developers' Use of Coding Agents. *Proceedings of the 23rd International Conference on Mining Software Repositories, MSR '26*, 520-532. https://doi.org/10.1145/3793302.3793377

Fawzy, A., Tahir, A., & Blincoe, K. (2026,). *From Prompting to Verification: How Experience Shapes Vibe Coding Practices*. https://arxiv.org/abs/2605.24521

Heesch, U. van, Zimmermann, O., & Kohls, C. (2026,). *What Will Remain Human in Software Architecture? A Focus Group Report*. https://arxiv.org/abs/2609.30334

Jackson, V., Cha, Y., Prikladnicki, R., & Hoek, A. van der. (2026). The Role of LLMs in Collaborative Software Design. *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering, FSE Companion '26*, 1783-1792. https://doi.org/10.1145/3803437.3806702

Kang, T., Park, H., & Shin, S. (2026). The Role of Software Engineering Knowledge in Shaping Effective AI-Driven Development. *Proceedings of the 2026 9th International Conference on Information and Computer Technologies, ICICT 2026*, 417-423. https://doi.org/10.1145/3803291.3803317

Karroumi, Y., El Khattabi, K., & Ou-zennou, F. (2026). CICAP: A Human-AI Co-creation Paradigm for Ethical and Efficient Plugin Development in Moodle. *International Journal of Information and Education Technology*, *16*, 1869-1880. https://doi.org/10.18178/ijiet.2026.16.7.2649

Khojah, R., Oliveira Neto, F. G. de, & Leitner, P. (2024,). *From Human-to-Human to Human-to-Bot Conversations in Software Engineering*. https://arxiv.org/abs/2405.12712

Klatt, F., Grazia, L. D., & Schneider, J. (2026,). *A-SAW: The Agentic Software Architecture Workflow*. https://arxiv.org/abs/2610.13647

Kshetri, N., & Voas, J. (2026). Is Vibe Coding the Future of Software?. *Computer*, *59*(3), 100-104. https://doi.org/10.1109/MC.2025.3634712

Kuutila, M., Mäntylä, M. V., Claes, M., & Elovainio, M. (2017). Reviewing Literature on Time Pressure in Software Engineering and Related Professions: Computer Assisted Interdisciplinary Literature Review. *2017 IEEE/ACM 2nd International Workshop on Emotion Awareness in Software Engineering (SEmotion)*, 54-59. https://doi.org/10.1109/SEmotion.2017.11

Muratovic, F., Kearns-Manolatos, D., & Alibage, A. (2025). Generative AI in Software Development: Challenges, Opportunities, and New Paradigms for Quality Assurance. *Computer*, *58*(7), 31-39. https://doi.org/10.1109/MC.2025.3556330

Salomon, M., Koshchenko, E., Sergeyuk, A., Holmes, R., Murphy, G. C., & Fritz, T. (2026). From Disruptions to Discussions: How GenAI Impacts Human Interactions in Software Development. *IEEE Transactions on Software Engineering*, *52*(7), 2095-2110. https://doi.org/10.1109/TSE.2026.3655626

Srinivas, P., Rahmath Mohammad, A., Neelima, T., Kundiladi, M. S., Ramamoorthy, S., & Katta, S. (2026). Generative AI-Assisted Framework for Secure and Autonomous Software Development Lifecycle Optimization. *2026 6th International Conference on Image Processing and Capsule Networks (ICIPCN)*, 1700-1705. https://doi.org/10.1109/ICIPCN67432.2026.11438791

Tabarsi, B., Reichert, H., Gilson, S., Limke, A., Kuttal, S., & Barnes, T. (2026). LLMs' reshaping of people, processes, products, and society in software development: a qualitative exploration with early adopters. *Empirical Software Engineering*, *32*(1). https://doi.org/10.1007/s10664--026-10917-0

Villani, T. S., & Kellogg, M. (2026). Command Coding: A Methodological Framework for AI-Accelerated Software Development. *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering, ASE '26*, 4493-4503. https://doi.org/10.1145/3832783.3834536

Wang, Y., Li, R., Liang, P., Cai, Y., Li, Z., Shahin, M., Khan, A. A., & Feng, Q. (2026,). *Using LLMs in Software Design: An Empirical Study of GitHub and A Practitioner Survey*. https://arxiv.org/abs/2605.01392

Wu, Y., Li, Z. S., Stolee, K. T., & Xu, B. (2026). How Do Developers Interact with AI? An Exploratory Study on Modeling Developer Programming Behavior. *Proc. ACM Softw. Eng.*, *3*(FSE). https://doi.org/10.1145/3808120

Yu, L., Alégroth, E., Chatzipetrou, P., & Gorschek, T. (2026). A Framework for Evaluating GenAI Adoption and Use in Software Engineering. *IEEE Transactions on Software Engineering*, *52*(8), 2233-2249. https://doi.org/10.1109/TSE.2026.3688745

Zhang, S., Xing, Z., Guo, R., Xu, F., Chen, L., Zhang, Z., Zhang, X., Feng, Z., & Zhuang, Z. (2025). Empowering Agile-Based Generative Software Development through Human-AI Teamwork. *ACM Trans. Softw. Eng. Methodol.*, *34*(6). https://doi.org/10.1145/3702987